

XiaoLee — Registro Técnico (RT): Integração Stellar

Documento Vivo — ADR + Decisões de Produto + Arquitetura

Atualizado em: **2026-05-08** | Sprint 1 — Stellar (37 Graus em andamento) Progresso Stellar: [#.....] 10% — Fase 1 iniciada. Classificação: **AI Ops Layer sobre Stellar — MVP em construção, base técnica sólida.**

Índice

1. [O Que é a XiaoLee](#)
 2. [Por Que Stellar](#)
 3. [Para Quem](#)
 4. [O Que Estamos Ofertando](#)
 5. [Decisões de Produto \(PDRs\)](#)
 6. [Decisões de Arquitetura \(ADRs\)](#)
 7. [Stack Tecnológico — Stellar](#)
 8. [Arquitetura de Alto Nível](#)
 9. [Fluxos Críticos](#)
 10. [Contrato Soroban — XiaoLee Core](#)
 11. [XiaoLee como AI Ops Layer — OrchestrationService](#)
 12. [Roadmap por Fases \(37 Graus\)](#)
 13. [GTM Plan](#)
 14. [Métricas de Sucesso](#)
 15. [Riscos e Mitigações](#)
 16. [Pendências e Decisões em Aberto](#)
 17. [Histórico de Atualizações](#)
-

1. O Que é a XiaoLee

XiaoLee é uma **AI ops layer**: uma camada de inteligência que senta entre o usuário e a infraestrutura Stellar, orquestrando operações on-chain via linguagem natural. O usuário interage via Twitter DM, Telegram ou web — swaps, campanhas de criadores, distribuição de recompensas e pagamentos — sem precisar entender carteiras, slippage ou contratos.

O princípio central é **wallet-first e não-custodial**: o backend orquestra, prepara e roteia; a assinatura de qualquer transação fica sempre com o usuário na própria carteira.

A integração Stellar expande esse protocolo para o ecossistema Stellar, aproveitando a infraestrutura nativa de pagamentos (EtherFuse, stablecoins, DEX) e os contratos Soroban.

2. Por Que Stellar

Motivação	Detalhe
Infraestrutura de pagamentos real	Stellar tem âncoras reguladas que conectam fiat LATAM ↔ stablecoins ↔ DeFi. O programa 37 Graus tem parceria direta com EtherFuse como provedor de on/off ramp — acesso preferencial ao rail de pagamentos já está garantido.
Custo de transação	Fees em Stellar são frações de centavo. Viabiliza micropagamentos e recompensas pequenas (sem alto custo de gás).
Soroban	Smart contracts Rust/WASM com modelo de storage e auth moderno. Suficiente para o protocolo XiaoLee sem complexidade desnecessária.
Stellar DEX nativo	Order book + AMM na camada 1. Swaps sem precisar de DEX de terceiro com risco adicional de contrato.
37 Graus	Programa NearX/SDF com \$20k em prêmios, mentoria direta da Stellar Development Foundation e residência no Rio. Alinhamento estratégico direto.
Mercado BR desatendido	150M usuários Pix. Nenhuma interface AI conecta Pix ↔ DeFi hoje. XiaoLee preenche esse gap.

3. Para Quem

Segmento	Perfil	Dor Central
Criadores LATAM e globais	Influenciadores, streamers, artistas com audiência no Twitter/X — Brasil como ponto de entrada, expansão para toda a América Latina e mercado global	Não têm ferramenta para monetizar engajamento on-chain sem exigir que o fã configure carteira do zero
Fãs / usuários crypto-nativos	18–35 anos, ativos em DeFi globalmente, sem paciência para múltiplas UX diferentes	Cada protocolo tem interface diferente; querem uma conversa só — em qualquer língua, em qualquer chain
Usuários não-crypto	Qualquer pessoa com acesso a pagamento local (Pix no BR, outros rails na LATAM e globalmente) e vontade de participar de campanhas	Barreira de entrada para DeFi via payment rail local inexistente com XiaoLee
Projetos Web3 globais	Times que querem rodar campanhas sociais com recompensas on-chain em qualquer mercado	Custo alto de desenvolvimento customizado; XiaoLee entrega como plataforma escalável independente de região

4. O Que Estamos Ofertando

Produto Principal

Uma interface AI conversacional para o DeFi Stellar com três pilares:

1. Engine de Campanhas Criadores publicam campanhas com tarefas sociais (seguir, RT, comentar) e definem recompensas em \$XLEE, XLM ou USDC. O usuário completa as tarefas, a XiaoLee verifica e distribui a recompensa diretamente na carteira

Freighter — sem código, sem custódia.

2. DeFi Conversacional Swaps, consultas de saldo, envio de pagamentos — tudo por mensagem. "Troca 50 USDC por XLM" → XiaoLee apresenta quote → usuário confirma → Freighter assina → transação enviada via Stellar DEX.

3. Gateway de Pagamentos BR/LATAM On/off ramp via **EtherFuse** (parceiro oficial do 37 Graus). O usuário entra com Pix, opera no DeFi Stellar, e sai quando quiser — sem precisar comprar cripto em exchange. EtherFuse abstrai toda a complexidade de conversão fiat ↔ Stellar.

Diferenciais

- Linguagem natural como única interface — sem curva técnica
- Não-custodial — nenhuma chave privada toca o backend
- Pix nativo — único protocolo DeFi com esse caminho no Brasil
- Auditável — toda distribuição de recompensa registrada on-chain via Soroban
- Identidade social — Twitter ID como âncora de identidade cross-wallet

5. Decisões de Produto (PDRs)

PDR-001 — Twitter ID como identidade primária

Decisão	Twitter ID é o identificador único de cada usuário. A wallet Freighter é vinculada ao Twitter ID, não o contrário.
Alternativa rejeitada	Endereço de carteira como identidade primária.
Rationale	XiaoLee nasceu como assistente de Twitter DM. O vínculo social-financeiro é o diferencial do protocolo — uma campanha não recompensa "uma carteira", recompensa "um criador com audiência verificada". A wallet pode ser trocada; a identidade social não.
Consequência	<code>twitter_id</code> é a chave de busca no contrato Soroban, no DB e no sistema de campanhas.

PDR-002 — Freighter como wallet principal

Decisão	Freighter é a wallet padrão suportada no frontend.
Alternativa rejeitada	LOBSTR e xBull como primárias.
Rationale	Freighter é mantido pela Stellar Development Foundation, tem TypeScript SDK oficial, suporte nativo a Soroban, e é a wallet de referência para dApps no ecossistema Stellar. Melhor integração com contratos Soroban e documentação mais completa para desenvolvedores.
Futuro	Suporte a xBull e LOBSTR via abstração <code>walletAdapter</code> no frontend — adicionados sem quebrar a interface.

PDR-003 — On/off ramp via EtherFuse (Sprint 2)

Decisão	Integrar EtherFuse como provedor de on/off ramp para pagamentos BR/LATAM. EtherFuse é parceiro oficial do programa 37 Graus — acesso direto à API garantido para participantes.
Alternativa considerada	Integrar BRLA âncora diretamente via SEP-24 sem EtherFuse.
Rationale	EtherFuse abstrai a complexidade de SEP-24, KYC, compliance e conversão fiat ↔ Stellar. Com a parceria do programa, temos suporte técnico direto e onboarding facilitado. Reduz o escopo de implementação do Sprint 2 significativamente, aumentando a chance de entrega com qualidade no prazo. Pix como payment rail é o diferenciador de produto; EtherFuse é o mecanismo de implementação.
Consequência	Nenhuma integração direta com âncoras BRLA/Bitso necessária no Sprint 2. XiaoLee chama a API EtherFuse; o restante é transparente para o usuário.

PDR-004 — \$XLEE como Stellar Asset (SAC), não token Soroban puro

Decisão	Emitir \$XLEE como Stellar Asset nativo e implantar o Stellar Asset Contract (SAC) para interação programática via contratos Soroban.
Alternativa rejeitada	Implementar \$XLEE como token Soroban customizado (equivalente a um ERC-20).
Rationale	Stellar Assets têm liquidez nativa no Stellar DEX imediatamente após emissão, sem custo extra de auditoria de contrato customizado. O SAC provê exatamente a interface necessária para o contrato Soroban distribuir \$XLEE. Simplicidade arquitetural sem perda de funcionalidade. Complexidade adicional (vesting, locks) pode ser adicionada via Soroban wrapper quando necessário.

PDR-005 — SEP-10 para autenticação não-custodial

Decisão	Todo usuário autenticado na XiaoLee passa pelo fluxo SEP-10. O resultado é um JWT padrão utilizado em todas as chamadas autenticadas.
Alternativa rejeitada	Autenticar via assinatura de mensagem arbitrária (não-padrão em Stellar).
Rationale	SEP-10 é o padrão Stellar de autenticação. Implementar fora do padrão quebraria compatibilidade com âncoras e futuros integradores. O fluxo de challenge/response é bem documentado, seguro, e o Freighter o suporta nativamente.

PDR-006 — Soroban para o contrato XiaoLee Core

Decisão	O registro on-chain de recompensas de campanha é feito via contrato Soroban <code>xiaolee_core</code> , com deploy no Stellar Testnet no Sprint 1 e Mainnet no Sprint 3.

Alternativa considerada	Registrar recompensas apenas no banco de dados off-chain.
Rationale	A auditabilidade on-chain é um diferencial de produto — criadores e usuários podem verificar que as recompensas foram distribuídas sem depender de confiança no backend. Soroban foi escolhido por ser Rust/WASM com modelo de storage e auth moderno (<code>require_auth()</code>), integração nativa com Stellar Assets via SAC, e suporte direto da SDF.

PDR-007 — x402 para micropagamentos AI (Sprint 2+)

Decisão	Implementar suporte ao protocolo x402 (HTTP-native micropayments) nos endpoints de AI premium da XiaoLee.
Rationale	O 37 Graus menciona x402 explicitamente como entregável do Sprint 2. XiaoLee como agente AI que cobra micropagamentos em XLM/USDC por queries premium cria um modelo de negócio on-chain sustentável e demonstra caso de uso avançado de Stellar para agentic economy.

PDR-008 — Track A no programa 37 Graus

Decisão	Inscrever XiaoLee no Track A (produto com MVP em operação).
Rationale	A base do protocolo (backend, campanhas, AI, observabilidade) está funcional com 65+ testes. A integração Stellar é uma expansão de produto, não um rebuild do zero. Track A posiciona melhor para os critérios de escala do programa e é mais honesto com o estado atual do produto.

6. Decisões de Arquitetura (ADRs)

ADR-001 — StellarAdapter como camada de abstração de protocolo

Contexto: O backend FastAPI concentra toda a lógica de negócio (campanhas, AI, usuários, rate limit). A integração com os protocolos Stellar precisa ser isolada para permitir evolução independente e testes unitários.

Decisão: Criar uma classe `StellarAdapter` que encapsula toda a interação com a Stellar — Horizon, Soroban e EtherFuse. O `OrchestrationService` delega ao `StellarAdapter` conforme a intenção detectada pelo Gemini.

```
class StellarAdapter:
    async def get_balance(self, wallet: str, asset: str) -> BalanceResult: ...
    async def prepare_swap(self, wallet, from_asset, to_asset, amount) -> SwapQuote: ...
    async def send_payment(self, from_wallet, to_wallet, asset, amount) -> TxResult: ...
    async def verify_wallet_auth(self, wallet: str, signed_xdr: str) -> bool: ...
    async def record_reward(self, twitter_id: str, wallet: str, amount: float) -> TxResult: ...
    async def initiate_pix_deposit(self, wallet: str, amount_brl: float) -> DepositResult: ...
    async def initiate_pix_withdrawal(self, wallet: str, amount_brl: float) -> WithdrawResult: ...
```

Alternativa rejeitada: Lógica Stellar espalhada diretamente no OrchestrationService. Rejeitado por acoplamento total e impossibilidade de teste isolado.

ADR-002 — SEP-10 implementado no backend como middleware de auth

Contexto: A autenticação não-custodial padrão no ecossistema Stellar é o SEP-10 — prova que o usuário controla uma conta sem expor a chave privada.

Decisão: Dois endpoints no backend FastAPI:

- `GET /auth/stellar/challenge?account=G...` — retorna XDR de challenge transaction
- `POST /auth/stellar/token` — valida XDR assinado, emite JWT

O backend mantém um keypair dedicado exclusivamente para challenges SEP-10 (nunca usado para fundos). O JWT emitido é o único token de sessão da plataforma — todos os middlewares de autorização consomem o mesmo formato.

Fluxo completo:

1. Frontend: `GET /auth/stellar/challenge?account=G...ABC`
2. Backend: gera challenge XDR (`sequence=0`, `manage_data op`, `nonce`, `TTL 5min`)
3. Frontend: `Freighter.sign(XDR) → signed_xdr`
4. Frontend: `POST /auth/stellar/token { account, transaction: signed_xdr }`
5. Backend: valida assinatura com `stellar-sdk`, extrai public key
6. Backend: emite JWT `{ sub: twitter_id, stellar_wallet: G...ABC, chain: "stellar" }`
7. Frontend: armazena JWT no `localStorage`, inclui em todas as requests

ADR-003 — Stellar DEX (Path Payments) para swaps

Contexto: Stellar possui um DEX nativo na camada-1 com order book e suporte a path payments — liquidez disponível sem depender de contratos de terceiros.

Decisão: Usar `pathPaymentStrictSend` e `pathPaymentStrictReceive` da Stellar SDK para swaps. O `StellarAdapter.prepare_swap()` consulta o melhor caminho via Horizon `/paths/strict-receive`, retorna a transação não assinada (XDR), e o frontend apresenta o quote para confirmação antes de assinar com Freighter.

Vantagens sobre DEX de terceiro:

- Liquidez nativa da rede — sem smart contract intermediário adicional
- API pública via Horizon — sem dependência de serviço externo
- Suporte a path finding automático (USDC → XLM pode passar por intermediários)

Complemento (Sprint 3+): Integrar Aquarius AMM para pares de baixa liquidez via Soroban invocation.

ADR-004 — Soroban `xiaolee_core` com storage persistent e auth por Address

Contexto: Soroban usa um modelo de storage e autenticação próprio que precisa ser adotado corretamente para garantir segurança e eficiência no protocolo.

Decisão: Usar `env.storage().persistent()` com chaves enum para state do protocolo e do usuário. Auth via `Address::require_auth()` sem workarounds.

Storage Schema:

Tipo	Chave	Uso
instance	DataKey::GlobalConfig	Config global do protocolo — vive junto com o contrato
persistent	DataKey::User(twitter_id)	State por usuário — TTL gerenciado pelo backend

Auth: Toda função admin chama `admin.require_auth()` diretamente. Se o invoker não for o admin registrado no `GlobalConfig`, a transação falha automaticamente na VM — sem lógica extra de validação.

Token: SAC via `token::Client::new(&env, &xlee_sac_address)` — o endereço do SAC \$XLEE é passado no `initialize()` e armazenado no `GlobalConfig`.

TTL de storage: Entries `persistent` expiram no Stellar. O backend faz bump de TTL periodicamente para usuários ativos via `extend_ttl`.

ADR-005 — \$XLEE como Stellar Asset + SAC

Contexto: \$XLEE precisa existir no Stellar para campanhas distribuírem recompensas.

Decisão: Emitir \$XLEE como Stellar Asset (`XLEE:G...<issuer>`) e implantar o Stellar Asset Contract (SAC) via `stellar contract asset deploy`. O contrato Soroban usa o SAC para transferir \$XLEE aos usuários após verificação de campanha.

Setup sequencial:

1. Criar conta issuer dedicada (keypair separado, não o admin do contrato)
2. Emitir XLEE asset via Stellar SDK (`setOptions + changeTrust`)
3. `stellar contract asset deploy --asset XLEE:G...<issuer> --network mainnet`
4. Contrato Soroban recebe endereço SAC via `initialize()` e usa para transferências
5. Usuários estabelecem trustline antes de receber \$XLEE (Freighter prompta automaticamente)

UX do trustline: XiaoLee detecta ausência de trustline antes da claim e instrui o usuário via chat: "Para receber \$XLEE, você precisa adicionar a trustline. Quer que eu te guie?" O Freighter assina a operação de trustline com um clique.

ADR-006 — PostgreSQL com coluna `chain` para eventos multi-origem

Contexto: O banco de dados registra campanhas, participantes, swaps e usuários. Com Stellar, os mesmos modelos precisam suportar eventos de duas origens sem duplicar tabelas.

Decisão: Adicionar coluna `chain VARCHAR(16) DEFAULT 'stellar'` nas tabelas `campaign_participants` e `swap_records`. Adicionar campo `stellar_wallet VARCHAR(64)` em `user_profiles`. `twitter_id` permanece como FK primária em todos os modelos.

Alembic migration delta:

```
ALTER TABLE user_profiles ADD COLUMN stellar_wallet VARCHAR(64);
ALTER TABLE campaign_participants ADD COLUMN chain VARCHAR(16) DEFAULT 'stellar';
ALTER TABLE swap_records ADD COLUMN chain VARCHAR(16) DEFAULT 'stellar';
```

ADR-007 — Horizon event_stream para indexação de eventos on-chain

Contexto: Stellar expõe eventos on-chain em tempo real via Horizon SSE (Server-Sent Events), permitindo que o backend reaja a transações confirmadas sem polling.

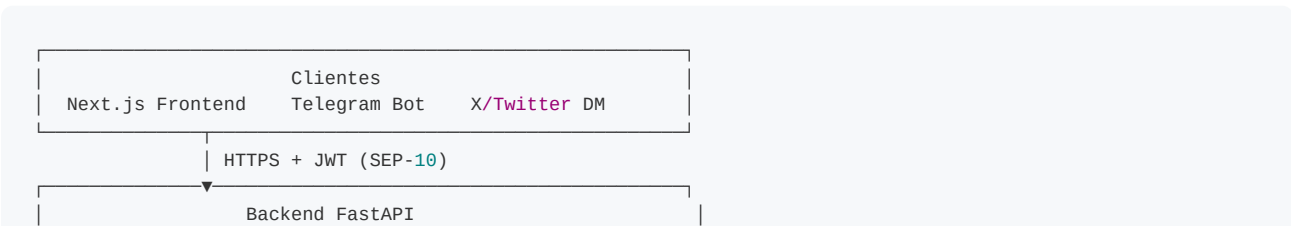
Decisão: Backend mantém uma goroutine/asyncio task que escuta `GET /accounts/{xiao_lee_admin}/transactions?cursor=now` via Horizon event_stream. Eventos de `record_reward` disparados pelo contrato Soroban são processados e persistidos no banco.

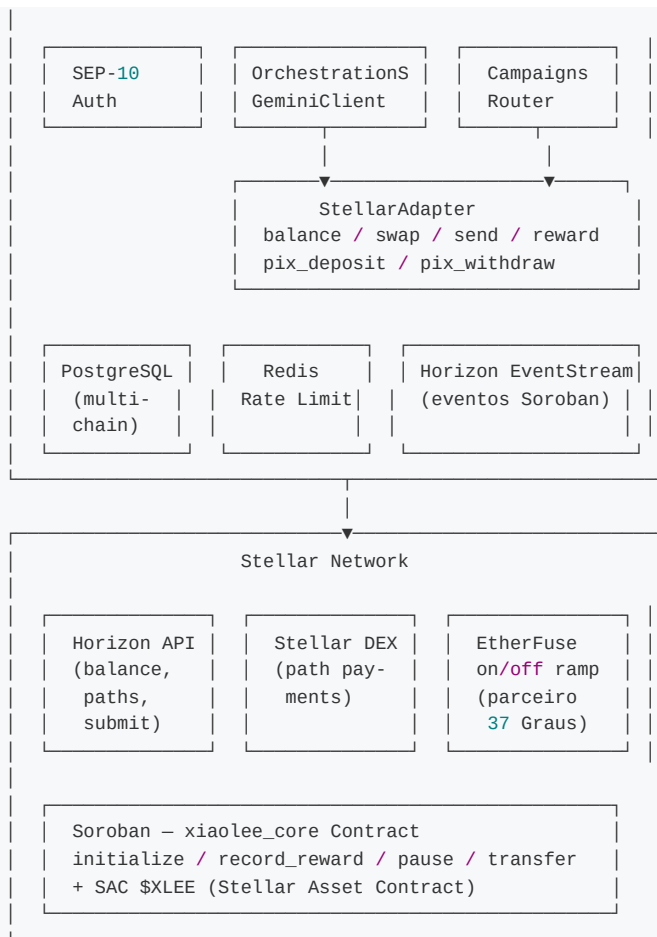
Alternativa considerada: Polling periódico via cron. Rejeitado por latência alta e custo de requests desnecessários.

7. Stack Tecnológico — Stellar

Camada	Tecnologia	Decisão
Smart Contract	Soroban SDK (Rust/WASM)	ADR-004
Wallet	Freighter + <code>@stellar/freighter-api</code>	PDR-002
Swaps	Stellar DEX via Horizon path payments	ADR-003
Token	Stellar Asset <code>xLEE</code> + SAC (SEP-41)	ADR-005 / PDR-004
Autenticação	SEP-10 challenge/response → JWT	ADR-002 / PDR-005
On/off ramp BR/LATAM	EtherFuse API (parceiro 37 Graus)	PDR-003
Micropagamentos AI	x402 protocol (XLM/USDC)	PDR-007
Eventos on-chain	Horizon event_stream (SSE)	ADR-007
AI (persona + intenção)	Gemini 1.5 Flash via GeminiClient	— (existente)
Backend	FastAPI (Python 3.12) + asyncio	— (existente, expandido)
Frontend	Next.js 15 (TypeScript)	— (existente, expandido)
Banco de dados	PostgreSQL 16 + Alembic async	ADR-006
Cache / Rate limit	Redis sliding window + fallback in-memory	— (existente)
Observabilidade	Prometheus <code>/metrics</code> + Grafana	— (existente, expandido com métricas Stellar)

8. Arquitetura de Alto Nível





9. Fluxos Críticos

9.1 Autenticação SEP-10 (não-custodial)

Usuário clica "Conectar Freighter"

- Frontend: GET /auth/stellar/challenge?account=G...ABC
- Backend: gera XDR challenge (sequence=0, manage_data, nonce, TTL 5min)
- Frontend: Freighter.sign(XDR) → signed_xdr
- Frontend: POST /auth/stellar/token { account, transaction: signed_xdr }
- Backend: valida assinatura com stellar-sdk
- Backend: emite JWT { sub: twitter_id, stellar_wallet: G...ABC }
- Frontend: armazena JWT – usuário está autenticado

9.2 Swap via Stellar DEX

Usuário: "troca 50 USDC por XLM"

- GeminiClient: intent = stellar_swap
- StellarAdapter.prepare_swap("USDC", "XLM", 50)
 - GET /paths/strict-send?source_asset=USDC&destination_asset=XLM
 - retorna: { destination_amount: 142.3 XLM, path: [XLM], fee: 0.00001 XLM }
- XiaoLee: "Você receberá ~142.3 XLM. Confirma?"
- Usuário confirma
- Frontend: constrói pathPaymentStrictSend XDR
- Freighter: usuário assina

- Frontend: submete via Horizon
- Backend: Horizon event_stream detecta tx → persiste swap_record

9.3 Campanha — Join / Verify / Claim (Stellar)

```
POST /campaigns/join { campaign_id, stellar_wallet, chain: "stellar" }
  → persiste participant com chain="stellar", status="enrolled"
  → 409 Conflict se já inscrito (UniqueConstraint)
```

Usuário completa tarefas sociais (follow, RT, comment)

```
POST /campaigns/verify { campaign_id, twitter_id, proof: { ... } }
  → Backend verifica tarefas via API Twitter/X
  → status → "tasks_verified"
```

```
POST /campaigns/claim { campaign_id, stellar_wallet, chain: "stellar" }
  → Backend valida JWT (SEP-10)
  → StellarAdapter.record_reward(twitter_id, stellar_wallet, amount)
    → SorobanClient invoca record_reward no contrato
    → SAC transfere $XLEE para stellar_wallet
  → Backend persiste receipt com tx_hash Stellar
  → Usuário recebe $XLEE no Freighter
```

9.4 On/off ramp via EtherFuse

Usuário: "quero depositar R\$ 100"

- GeminiClient: intent = onramp_deposit
- StellarAdapter.initiate_deposit(wallet=G..., amount=100, currency="BRL")
 - Chama EtherFuse API (parceiro 37 Graus)
 - EtherFuse retorna { deposit_url, transaction_id }
- Frontend exibe instrução de pagamento (Pix QR ou link)
- Usuário realiza o pagamento
- EtherFuse confirma → credita stablecoin na conta Stellar G...
- Horizon event_stream detecta → Backend notifica usuário
- XiaoLee: "Depósito confirmado! O que você quer fazer?"

Usuário opera: entra em campanha → completa tarefas → recebe \$XLEE

Usuário: "quero sacar"

- EtherFuse API handles offramp → usuário recebe em < 10min

EtherFuse abstrai a complexidade de SEP-24, KYC e conversão fiat ↔ Stellar. XiaoLee chama a API; o usuário só vê o resultado.

10. Contrato Soroban — XiaoLee Core

Instruções do Contrato

Instrução	Parâmetros	Descrição	Status
initialize	admin: Address, xlee_sac: Address	Inicializa GlobalConfig com admin e endereço SAC	Pendente
initialize_user	twitter_id: String	Registra usuário no storage persistent	Pendente

Instrução	Parâmetros	Descrição	Status
<code>record_reward</code>	<code>admin, twitter_id, amount: i128</code>	Registra e distribui recompensa \$XLEE	Pendente
<code>pause_protocol</code>	<code>admin</code>	Emergency pause — bloqueia todas as ops	Pendente
<code>unpause_protocol</code>	<code>admin</code>	Retoma operações	Pendente
<code>transfer_admin</code>	<code>admin, new_admin: Address</code>	Migra autoridade	Pendente

Storage Schema

```
// instance storage – sobrevive junto com o contrato
env.storage().instance().set(&DataKey::GlobalConfig, &GlobalConfig {
  admin: Address,
  xlee_sac: Address,    // endereço do SAC $XLEE
  paused: bool,
  version: u32,
  total_rewards: i128,
});

// persistent storage – TTL configurável, bump por usuário ativo
env.storage().persistent().set(&DataKey::User(twitter_id.clone()), &UserState {
  twitter_id: String,
  reward_count: u32,
  total_rewarded: i128,
});
```

Auth Pattern (Soroban)

```
pub fn record_reward(env: Env, admin: Address, twitter_id: String, amount: i128) {
  admin.require_auth(); // falha automaticamente se não autorizado
  let config: GlobalConfig = env.storage().instance().get(&DataKey::GlobalConfig).unwrap();
  require!(config.admin == admin, Error::Unauthorized);
  require!(!config.paused, Error::ProtocolPaused);

  // distribui via SAC
  let token = token::Client::new(&env, &config.xlee_sac);
  let user_address = Address::from_string(&env, &twitter_id); // ou mapeamento wallet
  token.transfer(&env.current_contract_address(), &user_address, &amount);
  // atualiza UserState + emite evento
}
```

Eventos Soroban

Evento	Campos	Quando
<code>reward_recorded</code>	<code>twitter_id, amount, total</code>	Cada recompensa distribuída
<code>protocol_paused</code>	<code>admin, timestamp</code>	Emergency pause
<code>protocol_unpaused</code>	<code>admin, timestamp</code>	Retomada
<code>admin_transferred</code>	<code>old_admin, new_admin</code>	Mudança de admin

Segurança

Item	Status
<code>require_auth()</code> em todas as funções admin	Pendente
Emergency pause no initialize	Pendente
Transfer admin implementado	Pendente
Overflow check em <code>total_rewarded (checked_add)</code>	Pendente
TTL bump para usuários ativos	Pendente
Auditoria externa	Fase 3
Admin → multisig Stellar nativo antes mainnet	Fase 3

11. XiaoLee como AI Ops Layer — OrchestrationService

O `OrchestrationService` existente detecta a chain pela sessão do usuário e injeta o contexto Stellar no prompt Gemini.

Novos Intents Stellar

Intent	Sprint	Descrição
<code>stellar_balance</code>	1	Consulta saldo XLM e assets via Horizon
<code>stellar_swap</code>	1	Quote + prepare pathPayment via Stellar DEX
<code>xlm_send</code>	1	Envio de XLM ou asset para outra wallet
<code>onramp_deposit</code>	2	Inicia depósito fiat via EtherFuse
<code>onramp_withdraw</code>	2	Inicia saque fiat via EtherFuse
<code>stablecoin_balance</code>	2	Consulta saldo de stablecoin + status de operação pendente
<code>xlm_price</code>	1	Cotação XLM/BRL e XLM/USD via CoinGecko
<code>xlee_balance</code>	2	Saldo \$XLEE + histórico de recompensas

Prompt de Sistema — Contexto Stellar

Quando `chain == "stellar"`, o `OrchestrationService` injeta:

```
[System: Chain ativa = Stellar. Wallet: G...ABC.
Saldo: 142.3 XLM | 50.00 USDC | 0.00 BRLA | 0.00 XLEE.
Operações: swap, send, pix_deposit, pix_withdraw, campanhas.
Responda em PT-BR. Apresente sempre o quote antes de executar.
Nunca execute transações sem confirmação explícita do usuário.]
```

Princípio de Confirmação (non-custodial)

A XiaoLee nunca executa uma transação financeira sem confirmação explícita:

- 1. Usuário solicita ação
- 2. XiaoLee apresenta resumo (quote, valor, destino, fee estimada)
- 3. Usuário confirma com "sim", "pode ir" ou similar
- 4. Frontend solicita assinatura no Freighter
- 5. Transação enviada — apenas após assinatura do usuário

12. Roadmap por Fases (37 Graus)

Programa NearX/Stellar: 04/05/2026 a 11/06/2026 | 5 Sprints | Residência Rio 08-11/06. Track A — produto com MVP em operação, foco em escala Stellar.

Fase 1 — Infraestrutura Stellar + Primeiros Usuários

Período: 04/05 – ~11/05 | **Meta do Programa:** domínio da infraestrutura Stellar + primeiros usuários

Entregável	Status	Prioridade
StellarAdapter base: <code>get_balance</code> , <code>prepare_swap</code> , <code>send_payment</code>	Em andamento	P0
SEP-10 auth: <code>/auth/stellar/challenge</code> + <code>/auth/stellar/token</code>	Pendente	P0
Freighter wallet connect no frontend	Pendente	P0
Alembic migration: coluna <code>chain</code> + <code>stellar_wallet</code>	Pendente	P0
Deploy XiaoLee Core Soroban no Testnet	Pendente	P0
Novos intents: <code>stellar_balance</code> , <code>stellar_swap</code> , <code>xlm_send</code> , <code>xlm_price</code>	Pendente	P1
Horizon event_stream para indexação de eventos	Pendente	P1
Emissão do asset <code>XLEE</code> no Testnet + deploy SAC	Pendente	P1
Grafana: métricas Stellar (swaps, balance queries, auth success rate)	Pendente	P2

Meta de usuários: 50 wallets Freighter conectadas

Progresso: [#.....] 10%

Fase 2 — Pagamentos: Pix, Stablecoins, x402

Período: ~11/05 – ~18/05 | **Meta do Programa:** integrações avançadas de pagamento

Entregável	Status	Prioridade
EtherFuse API: <code>onramp_deposit</code> e <code>onramp_withdraw</code>	Pendente	P0
Intents on/off ramp no OrchestrationService	Pendente	P0
Suporte USDC e stablecoins via EtherFuse no StellarAdapter	Pendente	P0
Swap BRLA ↔ XLM via Stellar DEX	Pendente	P0

Entregável	Status	Prioridade
UX trustline \$XLEE: detecção + instruções via chat	Pendente	P1
x402 endpoint: <code>/v1/ai/query</code> aceita micropagamento XLM/USDC	Pendente	P1
Campanha com recompensa em BRLA (além de \$XLEE)	Pendente	P2

Meta: 10 transações Pix reais + 200 usuários ativos

Progresso: [.....] 0%

Fase 3 — DeFi e Protocolo

Período: ~18/05 – ~25/05 | **Meta do Programa:** DeFi e construção sobre protocolos

Entregável	Status	Prioridade
XiaoLee Core Soroban — deploy em Mainnet	Pendente	P0
Emissão \$XLEE Mainnet + deploy SAC Mainnet	Pendente	P0
<code>record_reward</code> real em Mainnet	Pendente	P0
Campanhas com distribuição on-chain verificável	Pendente	P0
Aquarius AMM integration para pares de baixa liquidez	Pendente	P1
Testes de carga: endpoints Stellar (p95 < 500ms)	Pendente	P1
<code>/health/detailed</code> inclui Stellar RPC, Horizon, BRLA âncora	Pendente	P1
Revisão de segurança do contrato Soroban (community/Discord SDF)	Pendente	P2

Meta: 5 criadores com campanhas Stellar ativas + 500 completions on-chain

Progresso: [.....] 0%

Fase 4 — GTM e Narrativa de Investidor

Período: ~25/05 – ~01/06 | **Meta do Programa:** GTM + pitch preparation

Entregável	Status	Prioridade
Deck de pitch (produto + métricas reais + tração)	Pendente	P0
One-pager para investidores	Pendente	P0
Vídeo demo: fluxo Pix → campanha → saque (< 3min)	Pendente	P0
Case de criador BR: campanha real com métricas publicadas	Pendente	P0
RT atualizado com dados reais de tração	Pendente	P1
Thread Twitter/X: resultado dos sprints com números reais	Pendente	P1

Meta: 1.000 usuários únicos + 50 tx Pix + 3 criadores ativos + deck aprovado

Progresso: [.....] 0%

Fase 5 — Pitch Rio de Janeiro

Período: 08/06 – 11/06 | Evento presencial

Item	Descrição
Pitch presencial	5 min de apresentação + Q&A com investidores
Demo ao vivo	Fluxo Pix → campanha → saque no stage
Métricas para pitch	DAU, swap volume, Pix volume, campaign completions
Narrativa central	"XiaoLee é a interface AI para DeFi Stellar com Pix nativo"

13. GTM Plan

Posicionamento

XiaoLee é o primeiro protocolo DeFi conversacional com Pix nativo para a economia de criadores brasileira, operando sobre Stellar.

Canais de Aquisição

Canal	Tática	Fase	Meta
Twitter/X (dogfooding)	Campanha self-referencial: "complete tarefas → ganhe \$XLEE" na própria plataforma	1	500 usuários
Telegram	Bot XiaoLee com onboarding Stellar no grupo da comunidade	1-2	200 usuários
Criadores BR	Parceria com 3-5 influenciadores crypto BR para lançar campanhas pagas	2	1.000 participações
Comunidade Stellar BR	Presença ativa em grupos Telegram/Discord Stellar BR	1	Brand awareness
Build in public	Thread semanal no Twitter com progresso real + métricas	1-5	Credibilidade

Founder-Led Growth

O programa 37 Graus exige crescimento de usuários paralelo ao desenvolvimento desde o Sprint 1. Cadência:

- Início de cada sprint:** tweet anunciando o que será construído
- Mid-sprint:** update com screenshot ou demo em vídeo curto
- Fim de sprint:** resultado com métricas reais (usuários, txs, volume)
- Feature launch:** cada nova feature vira uma campanha na própria XiaoLee

Pitch para Criadores (30 segundos)

"Configure uma campanha em 10 minutos. Seus fãs completam tarefas sociais e recebem \$XLEE direto na carteira. Você paga quando as tarefas são verificadas. Sem código, sem custódia, auditável on-chain. Fãs que não têm cripto entram via Pix."

Modelo de Negócio

Fonte	Mecanismo	Ativação
Fee de swap	0.3% sobre volume de swaps executados via XiaoLee	Fase 3 (Mainnet)
Fee de campanha	0.5% do volume de recompensas distribuídas	Fase 3
x402 premium	Micropagamento XLM/USDC por queries AI avançadas	Fase 2+

14. Métricas de Sucesso

Produto (KPIs por fase)

Métrica	Fase 1	Fase 2	Pitch (Fase 4)
Wallets Freightner conectadas	50	200	500
Usuários únicos totais	50	300	1.000
Transações Pix	0	20	50
Campaign completions on-chain	0	500	2.000
Swap volume Stellar	0	5.000 XLM	20.000 XLM
Criadores ativos	0	2	5+
DAU/MAU ratio	—	> 15%	> 25%

SLAs Técnicos

Métrica	Target
p50 response time	< 200ms
p95 response time	< 500ms
Uptime	> 99.5%
SEP-10 auth success rate	> 99%
Swap execution success rate	> 95%
Pix deposit confirmation time	< 10 min

15. Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
EtherFuse API indisponível ou lenta	Baixa	Alto	Parceiro oficial 37 Gaus tem SLA; implementar timeout com mensagem clara ao usuário e retry com backoff
Freighter API breaking change	Baixa	Médio	Abstrair toda interação via <code>FreighterService</code> — nunca chamada direta de componente
Bug no contrato Soroban em Mainnet	Média	Alto	<code>pause_protocol</code> implementado desde o deploy; não distribuir \$XLEE real sem testes completos
Liquidez insuficiente no Stellar DEX para algum par	Alta	Médio	Verificar path antes de apresentar quote; nunca mostrar quote sem validar liquidez
Usuário sem trustline \$XLEE ao tentar receber recompensa	Alta	Médio	Detectar trustline antes da claim; XiaoLee instrui via chat e guia o usuário
TTL expirado no storage Soroban	Média	Médio	Backend faz <code>extend_ttl</code> periodicamente para usuários com atividade recente
KYC exigido pela EtherFuse para volumes maiores	Média	Médio	Mapear limites KYC da EtherFuse no início do Sprint 2 com o time do programa; comunicar limites no onboarding
Stellar network congestion	Baixa	Baixo	Retry com backoff exponencial no StellarAdapter; feedback ao usuário sobre delay

16. Pendências e Decisões em Aberto

ID	Pergunta	Prazo	Responsável
PD-001	EtherFuse: solicitar acesso à API e documentação técnica com o time do 37 Gaus; mapear limites KYC e disponibilidade de sandbox	Fase 1	Time
PD-002	\$XLEE: definir supply, política de emissão e mecanismo de distribuição antes do deploy em Mainnet	Fase 2	Decisão estratégica
PD-003	Auditoria do contrato Soroban: community review via Stellar Discord ou contratar auditora?	Fase 3	Time
PD-004	x402: implementar como feature do backend principal ou microserviço separado?	Fase 2	Arquitetura
PD-005	KYC para Pix acima de R\$ 500: BRLA exige verificação? Como tratar no fluxo XiaoLee?	Fase 2	Compliance
PD-006	Multisig para admin Soroban: usar multisig nativo Stellar (threshold keys) antes do Mainnet?	Fase 3	Segurança
PD-007	Armazenamento de wallet mapping (twitter_id → stellar_wallet): on-chain no Soroban ou somente off-chain no PostgreSQL?	Fase 1	Arquitetura

17. Histórico de Atualizações

Data	Versão	Mudanças
2026-05-08	1.0	Documento inicial — PDRs, ADRs, stack, arquitetura, fluxos, roadmap 37 Graus, GTM

XiaoLee — Protocolo DeFi Conversacional sobre Stellar.